

Firebase

Nada Yousry

Today's Roadmap

- Introduction to Backend & Cloud Computing
- Cloud Service Models (IaaS / PaaS / SaaS / BaaS)
- Why Firebase?
- Firebase Services Overview
- Setting Up Firebase with Flutter
- Firebase Authentication
- Cloud Firestore
- CRUD Operations
- Building the Study Planner App
- Firebase Pricing (Spark vs Blaze)
- Q&A





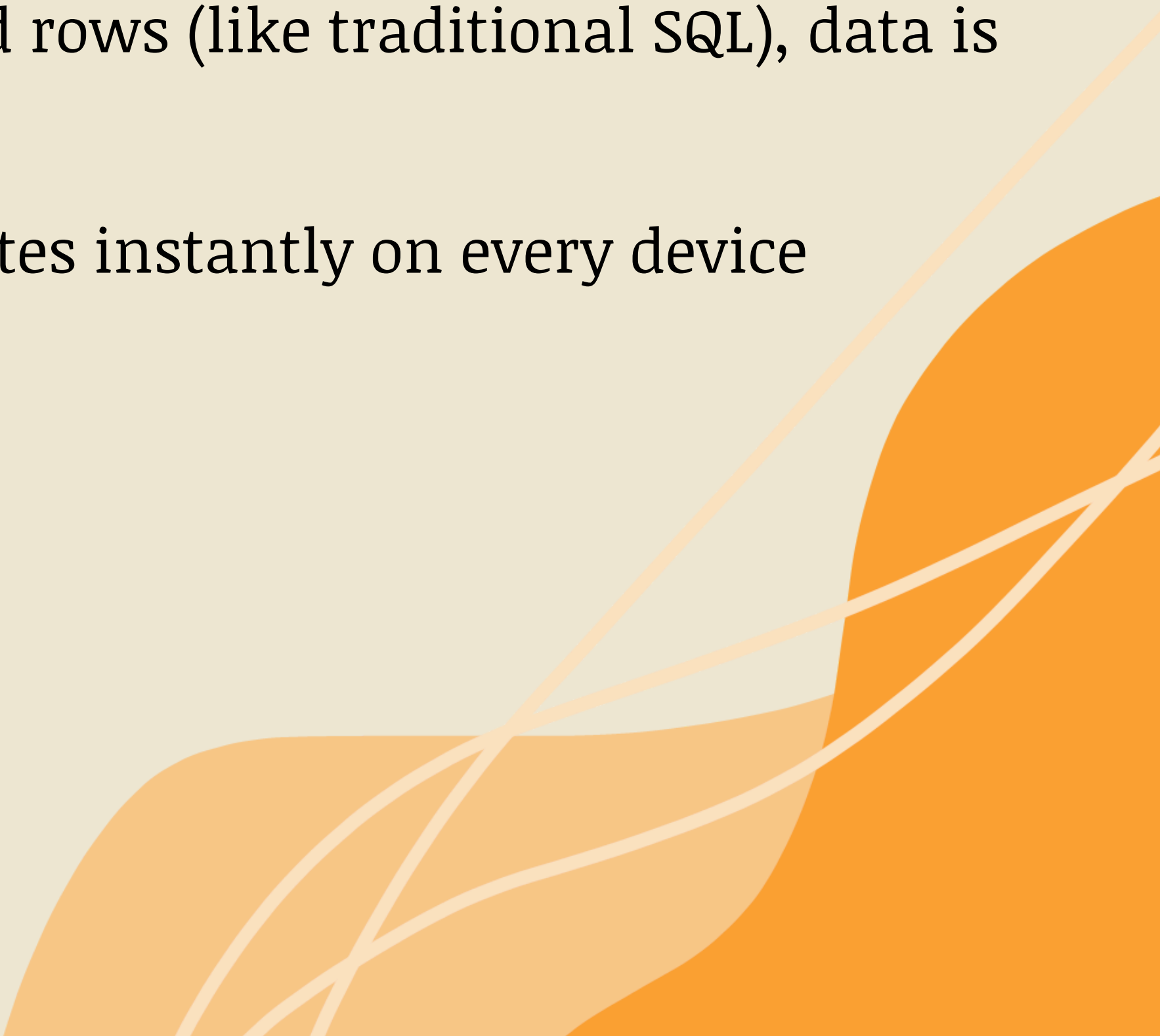
Firestore

Cloud Firestore

Firestore is Firebase's database — where we'll store all our Study Ritual tasks

It's a NoSQL database: instead of tables and rows (like traditional SQL), data is organized as Collections and Documents

It's real-time: any change to the data updates instantly on every device watching it — no refresh needed



Collections & Documents

Think of a Collection as a folder, and each Document inside it as a file with its own data.



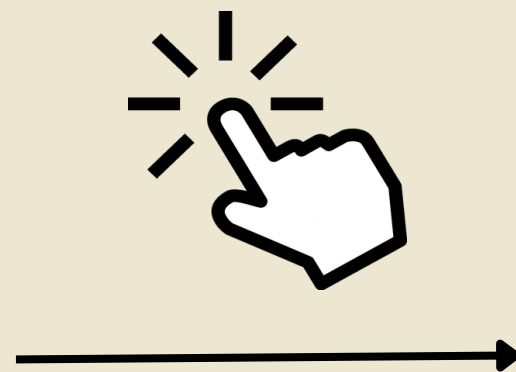
```
tasks (Collection)
├── task_1 (Document)
│   ├── title: "Study Math"
│   ├── isDone: false
│   └── createdAt: timestamp
└── task_2 (Document)
    ├── title: "Read Chapter 5"
    ├── isDone: true
    └── createdAt: timestamp
```



Tasks



task

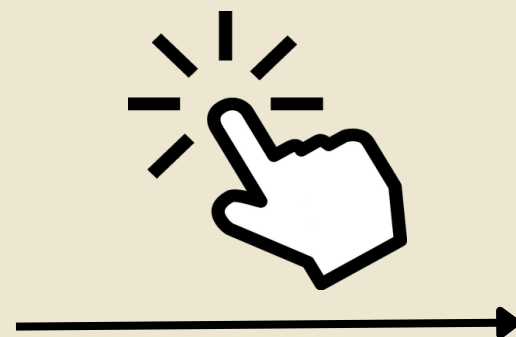


title : studying

.....

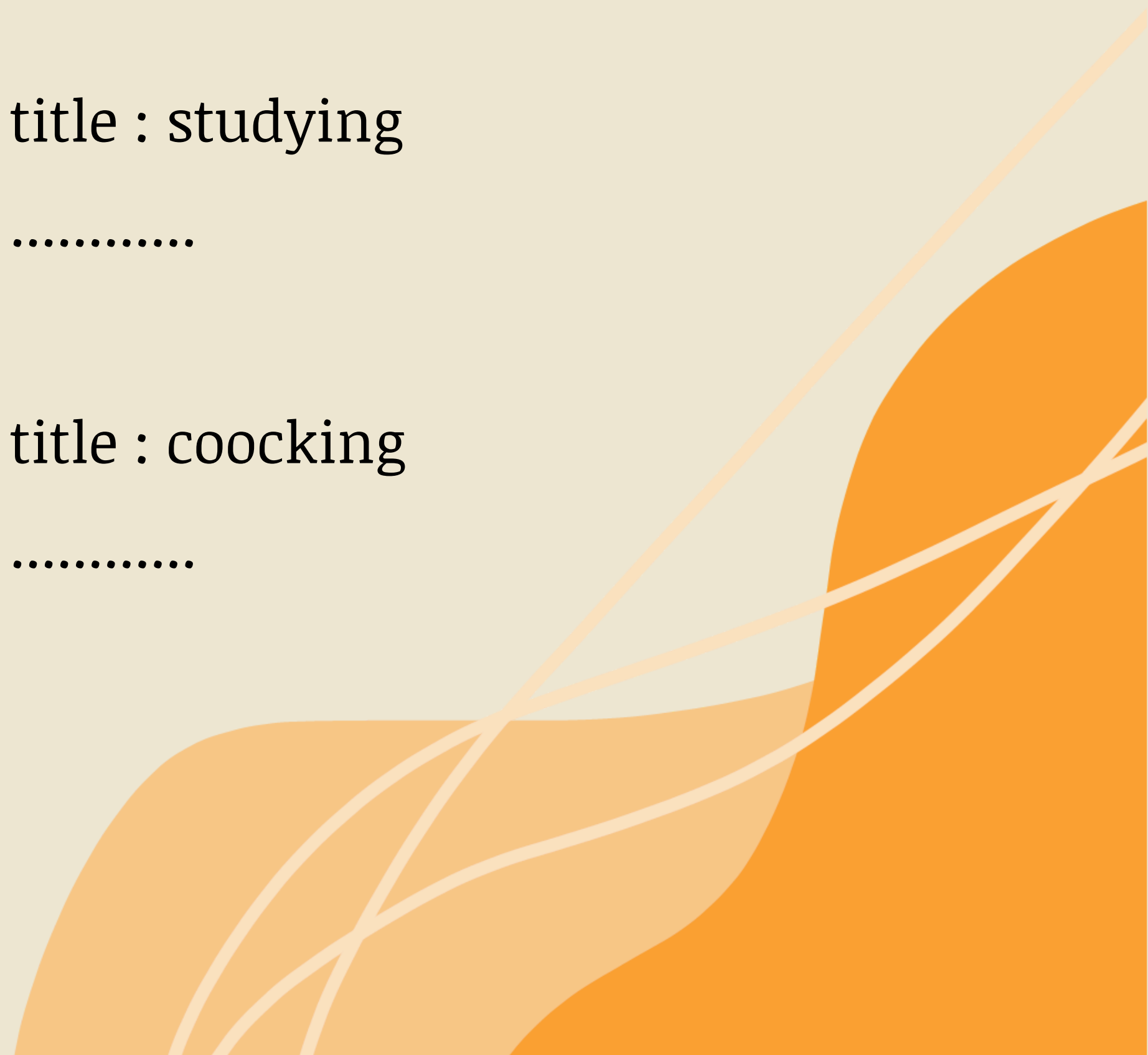


task

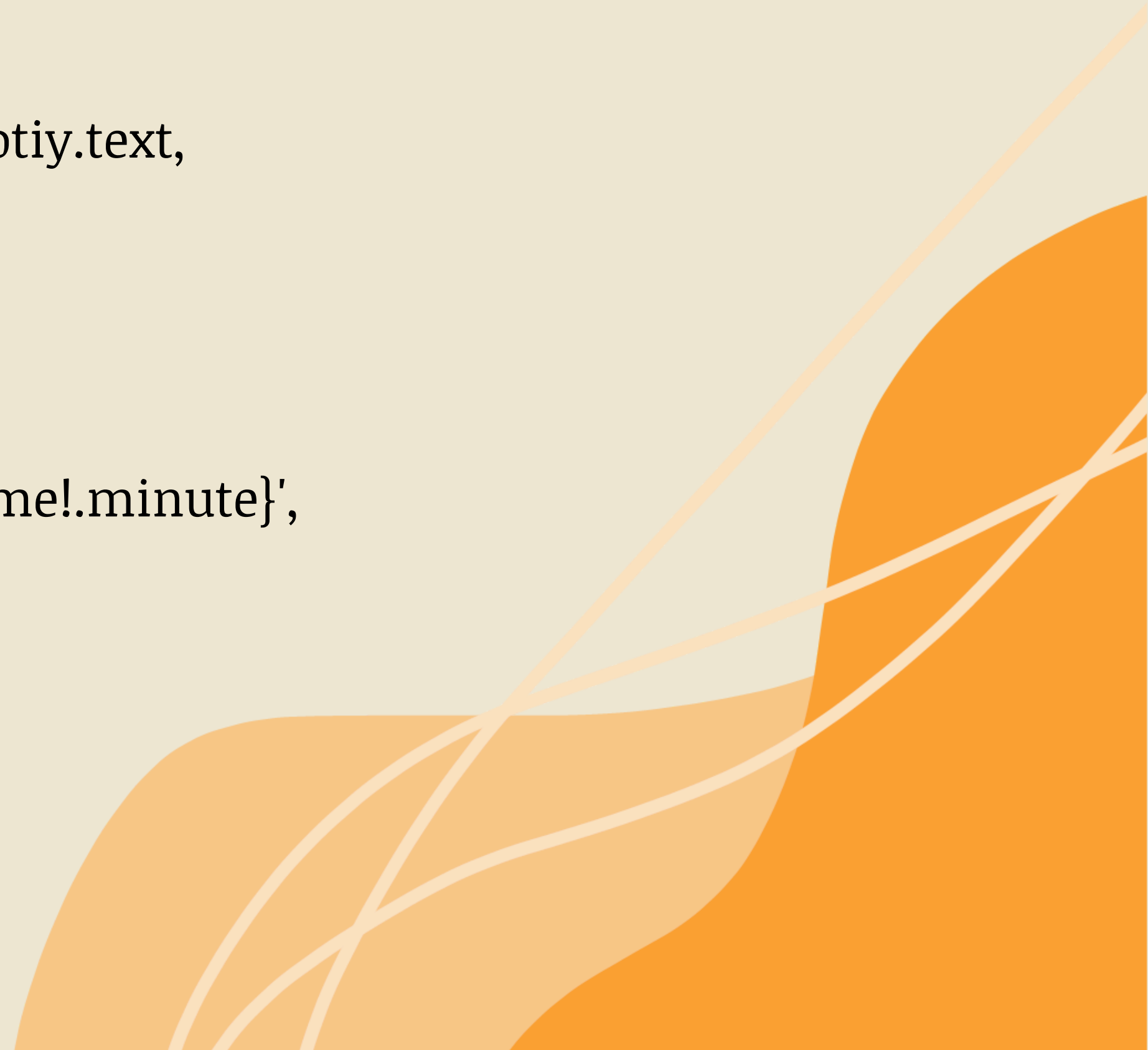


title : coocking

.....



```
await FirebaseFirestore.instance.collection("tasks").add({  
  'title': title.text.trim(),  
  'description': description.text.trim(),  
  'priority': priotiy.text.isEmpty ? 'Medium' : priotiy.text,  
  'isDone': false,  
  'createdAt': Timestamp.now(),  
  'dueDate': selectedDate,  
  'dueHour': '${selectedTime!.hour}:${selectedTime!.minute}',  
});
```

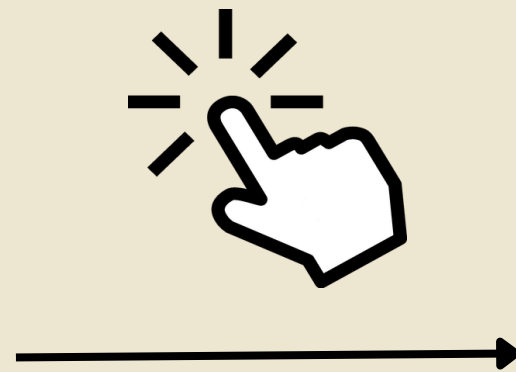




Tasks



task

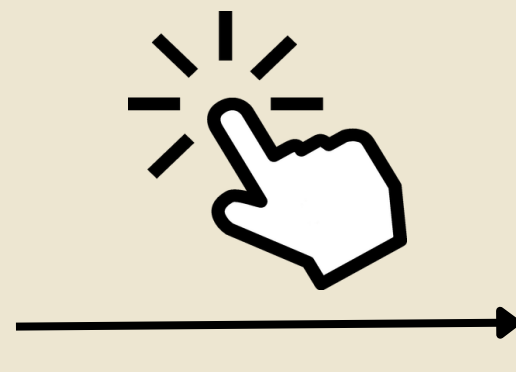


title : studying

.....



task



title : coocking

.....

Application : Please give me tasks of
user with ID kn5



Users



KN5



Tasks



task

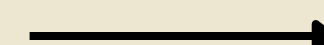


title : studying

.....



task



title : studying

.....



LB6



```
await FirebaseFirestore.instance
  .collection('users')
  .doc(userId)
  .collection("tasks")
  .add({
    'title': title.text.trim(),
    'description': description.text.trim(),
    'priority': priotiy.text.isEmpty ? 'Medium' : priotiy.text,
    'isDone': false,
    'createdAt': Timestamp.now(),
    'dueDate': selectedDate,
    'dueHour': '${selectedTime!.hour}:${selectedTime!.minute}',
  });
```





Tasks



task



title : studying

.....



task



title : coocking

.....

```
List <Task> tasks = snapShot Data  
[task , task , task , ..... ]
```

ListView

tasks[index]['title']

tasks[index]['subtitle']

tasks.removeAt(index)

[task , task ,.....]



tasks.removeAt(2)

task(2) deleted in app

task(0) deleted in server



Tasks



task



task



title : studying

.....

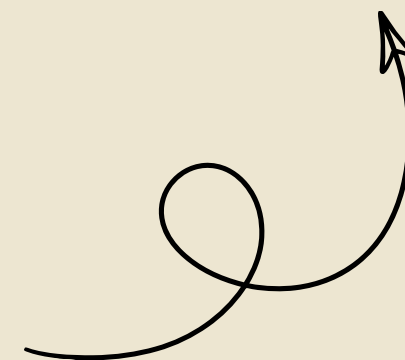
title : coocking

.....

into Properties

DID

DID



```
DocumentReference taskRef = await FirebaseFirestore.instance.collection('users')  
    .doc(userId).collection("tasks").doc();
```

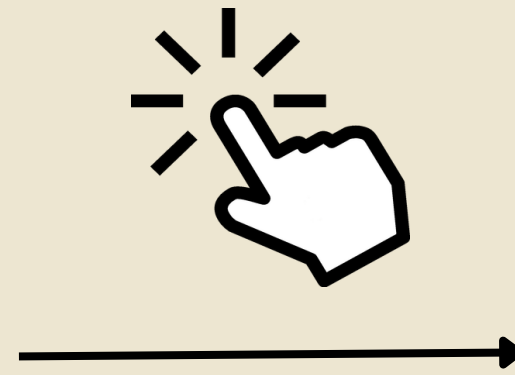
```
await taskRef.set({  
    'id': taskRef.id,  
    'title': title.text.trim(),  
    'description': description.text.trim(),  
    'priority': priotiy.text.isEmpty ? 'Medium' : priotiy.text,  
    'isDone': false,  
    'createdAt': Timestamp.now(),  
    'dueDate': selectedDate,  
    'dueHour': '${selectedTime!.hour}:${selectedTime!.minute}',  
});
```



Tasks



task

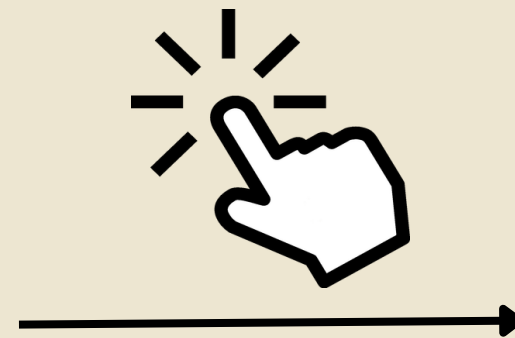


task_id : 5,
title : studying

.....



task



task_id : 6,
title : coocking

.....

CRUD Operations

Every interaction with Firestore falls into 4 operations:

- Create → add a new task
- Read → display existing tasks
- Update → mark a task as done, edit its title
- Delete → remove a task

These 4 operations are all we need to build almost any feature in Study Ritual

Notice the pattern: every operation starts the same way — `collection('tasks')` — then you say exactly what you want to do.



```
// Create
await
Firestore.instance.collection
('tasks').add({...});

// Read
Firestore.instance.collection
('tasks').snapshots()

// Update
await
Firestore.instance.collection
('tasks').doc(taskId).update({...});

// Delete
await
Firestore.instance.collection
('tasks').doc(taskId).delete();
```



What's Not So Good About Firebase?

Vendor Lock-in

Your data and app logic are tightly coupled to Google's ecosystem — migrating a large app to another backend later isn't easy or cheap

Cost Can Grow Fast

Free to start (Spark Plan), but as users and reads/writes scale up, the bill can climb quickly if you're not monitoring usage

Limited Complex Queries (Firestore specifically)

Firestore is NoSQL, so it doesn't support joins or complex relational queries like SQL — apps needing complex data relationships face structural challenges

Reliance on Internet Connectivity

Despite partial offline support, the core dependency on a stable internet connection is still there

Less Control Than a Custom Backend

You can't customize the infrastructure or control every detail the way you could with your own server



```
// Lazy Loading من غير  
ListView(children: posts.map((p) =>  
PostWidget(post: p)).toList())  
// حتى لو مقل هتظهر NOW كل العناصر بتتحمب وتتبنى
```

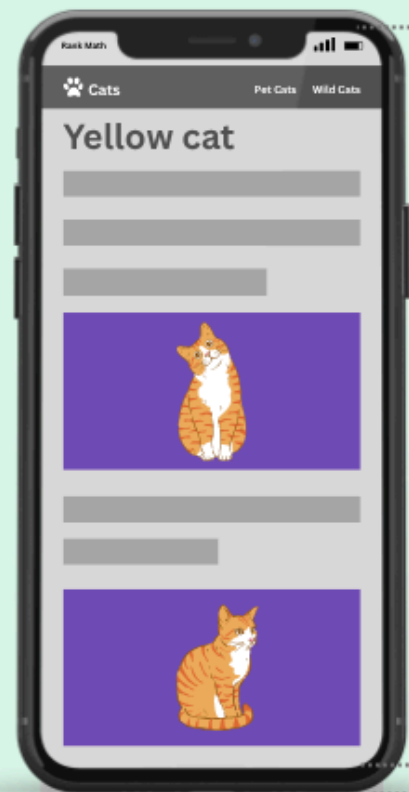
```
// Lazy Loading مع  
ListView.builder(  
  itemCount: posts.length,  
  itemBuilder: (context, index) =>  
PostWidget(post: posts[index]),  
)  
// العنصر بيتبنى بس وقت ما يحتاج يظهر فعليًا على  
الشاشة
```

How Lazy Loading Works

This webpage **uses** lazy loading

Images within the viewport are loaded

Images outside the viewport are **NOT** loaded



Content within the viewport

Content outside the viewport

This webpage **does not use** lazy loading

Images within the viewport are loaded

Images outside the viewport are loaded

